

awk

A note on names

awk is the original command

gawk is the GNU version of awk (we'll talk about GNU soon!)

On many modern Linux systems, using awk is really just using gawk in the background.

Running awk

```
awk -f program_file data_file
```

Or you can make a standalone script using a shebang:

```
#!/usr/bin/env gawk -f
```

awk

Data-driven scripting (e.g., data analysis)

awk

Data-driven scripting (e.g., data analysis)

Format of lines:

```
pattern {action}
```

awk

Data-driven scripting (e.g., data analysis)

Format of lines:

```
pattern {action}
```

Simple: Perform action on lines that match the pattern!

Terms

Given some data:

chevy	impala	1985	85	1550
ford	explor	2003	25	9500

Terms

Given some data:

						Records
chevy	impala	1985	85	1550		
ford	explor	2003	25	9500		

Terms

Given some data:

chevy	impala	1985	85	1550
ford	explor	2003	25	9500

Fields

Terms

Given some data:

Reference fields with \$1, \$2, or \$0 for whole line

chevy	impala	1985	85	1550
ford	explor	2003	25	9500

Fields

The simplest example

Before even reading the records, we can use the BEGIN and END patterns

The simplest example

Before even reading the records, we can use the BEGIN and END patterns

```
BEGIN {  
    print "Beginning now!"  
}  
END {  
    print "Ending now!"  
}
```

The simplest example

Before even reading the records, we can use the BEGIN and END patterns

Available as `step_0.awk` on GitHub

```
BEGIN {  
    print "Beginning now!"  
}  
END {  
    print "Ending now!"  
}
```

The simplest example

Before even reading the records, we can use the BEGIN and END patterns

Available as `step_0.awk` on GitHub

Run with: `awk -f step_0.awk cars.txt`

```
BEGIN {  
    print "Beginning now!"  
}  
END {  
    print "Ending now!"  
}
```

Pattern matching (step_01.awk)

```
BEGIN { print "Beginning now!" }
```

```
$1 == "ford" { print "Its a ford: " $0 }
```

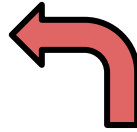
```
END { print "Ending now!" }
```

Pattern matching (step_01.awk)

```
BEGIN { print "Beginning now!" }
```

```
$1 == "ford" { print "Its a ford: " $0 }
```

```
END { print "Ending now!" }
```



Only runs on lines where first field (\$1) is "ford".

Remember \$0 is the whole line

Variables (step_02.awk)

```
BEGIN { print "Beginning now!" }

$1 == "ford" { print "Its a ford: " $0 }

$3 >= 2000 { count_new += 1 }

$3 < 2000 { count_old += 1 }

END {
    print "Number of new cars: " count_new
    print "Number of old cars: " count_old
    print "Ending now!"
}
```

Variables (step_02.awk)

```
BEGIN { print "Beginning now!" }  
$1 == "ford" { print "Its a ford: " $0 }  
$3 >= 2000 { count_new += 1 }  
$3 < 2000 { count_old += 1 }  
END {  
    print "Number of new cars: " count_new  
    print "Number of old cars: " count_old  
    print "Ending now!"  
}
```



We can do all our normal operations here (e.g., math)

Undeclared variables are 0 / empty string

Regex (step_03.awk)

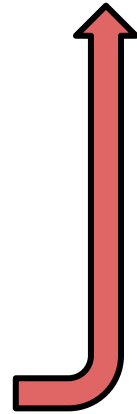
```
BEGIN { print "Beginning now!" }
$1 == "ford" { print "Its a ford: " $0 }
$3 >= 2000 { count_new += 1 }
$3 < 2000 { count_old += 1 }
$3 ~ /200[0-9]/ { print "Oughts detected: " $0 }
END {
    print "Number of new cars: " count_new
    print "Number of old cars: " count_old
    print "Ending now!"
}
```

Regex (step_03.awk)

```
BEGIN { print "Beginning now!" }
$1 == "ford" { print "Its a ford: " $0 }
$3 >= 2000 { count_new += 1 }
$3 < 2000 { count_old += 1 }
$3 ~ /200[0-9]/ { print "Oughts detected: " $0 }
END {
    print "Number of new cars: " count_new
    print "Number of old cars: " count_old
    print "Ending now!"
}
```

Checks to see if third field matches regex

Omit \$3 ~ to run regex on whole record (same as \$0 ~)



Advanced math (step_04.awk)

```
BEGIN { print "Beginning now!" }
$1 == "ford" { print "Its a ford: " $0 }
$3 >= 2000 { count_new += 1 }
$3 < 2000 { count_old += 1 }
$3 ~ /200[0-9]/ { print "Oughts detected: " $0 }
{ total_cost += $5 }
END {
    print "Number of new cars: " count_new
    print "Number of old cars: " count_old
    print "Average cost: " total_cost / NR
    print "Ending now!"
}
```

Advanced math (step_04.awk)

```
BEGIN { print "Beginning now!" }
$1 == "ford" { print "Its a ford: " $0 }
$3 >= 2000 { count_new += 1 }
$3 < 2000 { count_old += 1 }
$3 ~ /200[0-9]/ { print "Oughts detected: " $0 }
{ total_cost += $5 }
END {
    print "Number of new cars: " count_new
    print "Number of old cars: " count_old
    print "Average cost: " total_cost / NR
    print "Ending now!"
}
```

 No pattern? Runs for every record

Advanced math (step_04.awk)

```
BEGIN { print "Beginning now!" }
$1 == "ford" { print "Its a ford: " $0 }
$3 >= 2000 { count_new += 1 }
$3 < 2000 { count_old += 1 }
$3 ~ /200[0-9]/ { print "Oughts detected: " $0 }
{ total_cost += $5 }
END {
    print "Number of new cars: " count_new
    print "Number of old cars: " count_old
    print "Average cost: " total_cost / NR
    print "Ending now!"
}
```



No pattern? Runs for every record



NR = number of records processed **SO FAR**

Associative arrays (step_05.awk)

```
BEGIN { print "Beginning now!" }
$1 == "ford" { print "Its a ford: " $0 }
$3 >= 2000 { count_new += 1 }
$3 < 2000 { count_old += 1 }
$3 ~ /200[0-9]/ { print "Oughts detected: " $0 }
{ total_cost += $5 }
{ makes[$1] += 1 }
END {
    print "Number of new cars: " count_new
    print "Number of old cars: " count_old
    print "Average cost: " total_cost / NR
    for (key in makes) { print "Found " makes[key] " " key }
    print "Ending now!"
}
```

Associative arrays (step_05.awk)

```
BEGIN { print "Beginning now!" }
$1 == "ford" { print "Its a ford: " $0 }
$3 >= 2000 { count_new += 1 }
$3 < 2000 { count_old += 1 }
$3 ~ /200[0-9]/ { print "0ughts detected: " $0 }
{ total_cost += $5 }
{ makes[$1] += 1 }
END {
    print "Number of new cars: " count_new
    print "Number of old cars: " count_old
    print "Average cost: " total_cost / NR
    for (key in makes) { print "Found " makes[key] " " key }
    print "Ending now!"
}
```

 Use field 1 as our key

Associative arrays (step_05.awk)

```
BEGIN { print "Beginning now!" }
$1 == "ford" { print "Its a ford: " $0 }
$3 >= 2000 { count_new += 1 }
$3 < 2000 { count_old += 1 }
$3 ~ /200[0-9]/ { print "0ughts detected: " $0 }
{ total_cost += $5 }
{ makes[$1] += 1 }
END {
    print "Number of new cars: " count_new
    print "Number of old cars: " count_old
    print "Average cost: " total_cost / NR
    for (key in makes) { print "Found " makes[key] " " key }
    print "Ending now!"
}
```



Use field 1 as our key



Iterate over all keys in our associative array

Helpful links

List of variables to control awk:

https://www.gnu.org/software/gawk/manual/html_node/User_002dmodified.html

List of variables awk sets for you:

https://www.gnu.org/software/gawk/manual/html_node/Auto_002det.html

Simple intro to awk: <https://evodify.com/awk-is-awesome/>